

Модуль автомасштабирования интеллектуальных модулей для платформы MLOps

Спирин И. В., Крупнов И. И.

4 ноября 2024 г.

Аннотация

В этом реферате рассматривается разработка модуля автоматического масштабирования (автоскейлинга) для интеллектуальных сервисов, разработанного для использования на платформе MLOps. Этот модуль предоставляет простой и гибкий интерфейс для управления ресурсами, а также поддерживает гибернацию с автоматическим восстановлением.

Содержание

1 Введение	1
1.1 Предпосылки	1
1.2 Постановка задачи	1
2 Архитектура	2
2.1 Структурная схема	2
3 Алгоритм решения	3
3.1 Масштабирование	3
3.1.1 Сервер метрик и оповещения	3
3.1.2 Масштабирование с пользовательскими параметрами	3
3.1.3 Конфигурация масштабирования	3
3.1.4 Гибернация и восстановление	4
4 Тестирование	5
4.1 Протокол тестирования	5
4.1.1 Тестирование гибернации и восстановления	5
4.1.2 Тестирование масштабирования Deployment	6
4.2 Параметры тестирования	6
5 Ссылки на репозитории	6

1 Введение

1.1 Предпосылки

Интеллектуальные модули, развертываемые на платформе, нередко нагружают вычислительные, сетевые и дисковые ресурсы в процессе своей работы. Причем, такое поведение наблюдается и в режиме простоя, когда модуль не производит полезной нагрузки. Это приводит к неэффективному использованию «железа», а также может затронуть производительность сторонних программ.

Для решения этой проблемы используется автоматическое масштабирование, которое снижает финансовые затраты как клиентов, так и поставщиков облачных услуг.

Оркестратор Kubernetes уже предоставляет инструменты для автоскейлинга, такие как Horizontal Pod Autoscaler (HPA), Vertical Pod Autoscaler (VPA) и Kubernetes Event-driven Autoscaling (KEDA). Однако, они сложны в настройке и не интегрируются с платформой напрямую. В частности, при гибернации (полном выключении) сервиса, необходимо реализовывать обратное восстановление вручную для каждого компонента.

1.2 Постановка задачи

Необходимо разработать модуль автоскейлинга для интеллектуальных сервисов, который будет использоваться на платформе MLOps. Он должен отслеживать использование ресурсов пользовательскими приложениями и автоматически изменять количество реплик по значениям метрики, указанной пользователем.

В качестве целевого объекта для масштабирования могут использоваться Service, Deployment или ресурс MLOps-платформы MLComponent.

Также должна быть доступна возможность гибернации сервиса с автоматическим пробуждением через стандартный API. При поступлении HTTP-запроса на гибернированное приложение, оно должно восстанавливаться; при этом, до готовности приложения к работе, запросы будут удерживаться, а при превышении таймаута - должно возвращаться перенаправление (HTTP 307) на тот же URL.

Параметры таймаута и количества перенаправлений автоматически определяются по User-Agent клиента.

2 Архитектура

Модуль автомасштабирования будет выполнен в виде отдельного сервиса Kubernetes, написанного на языке Python со следующими библиотеками:

- FastAPI — для реализации асинхронного HTTP-сервера и API.
- kubernetes_asyncio — для взаимодействия с API Kubernetes в асинхронном режиме.
- APScheduler — для создания фоновых задач на сбор метрик и масштабирование.

Приложение будет состоять из веб-сервера FastAPI, который будет принимать запросы, переадресованные с гибернированных сервисов, и запросы с менеджера оповещений. Также, в фоновом режиме будет работать планировщик задач APScheduler, в который будут добавляться задачи на запрос метрик из сервера метрик с последующей их обработкой.

2.1 Структурная схема

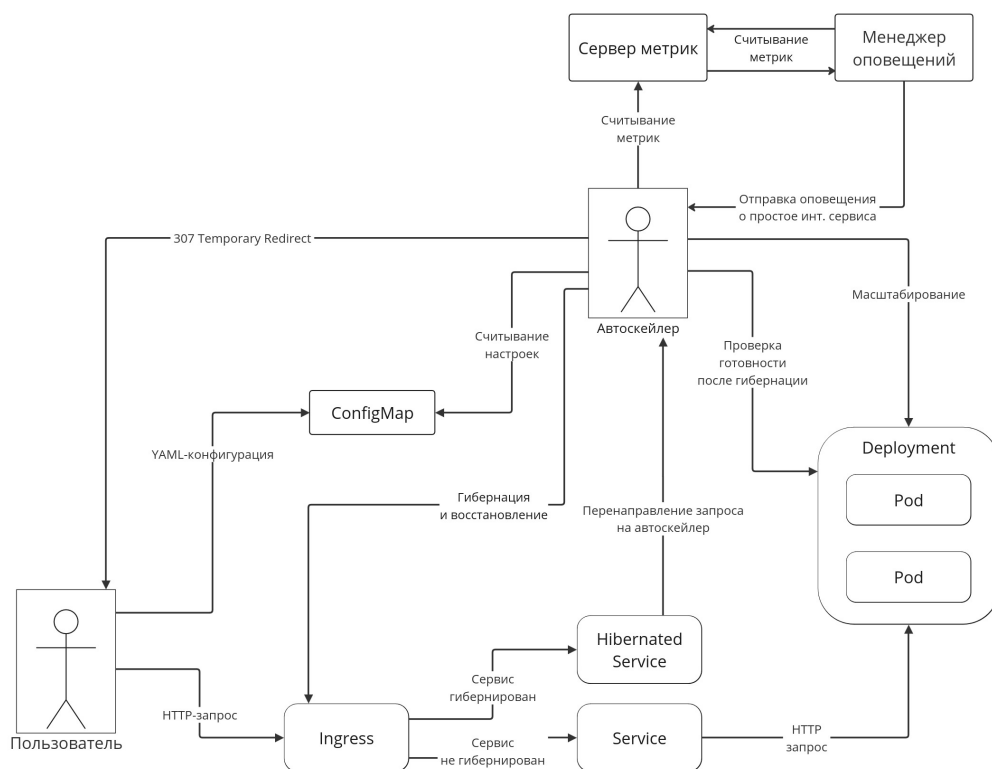


Рис. 1: Структурная схема модуля автоскейлинга

Пользователь модуля посылает HTTP-запросы на API интеллектуального модуля, которые приходят в общий Ingress.

При автоматической гибернации сервиса, Ingress меняется так, что последующие запросы будут перенаправляться в автоскейлер.

Если приходит перенаправленный запрос, автоскейлер удерживает его и запускает восстановление, после чего меняет Ingress обратно и возвращает Redirect на тот же URL.

В фоновом режиме автоскейлер собирает метрики из сервера метрик по расписанию, указанному в ConfigMap. Он сравнивает значения с пороговыми, и при необходимости масштабирует Deployment интеллектуального модуля.

3 Алгоритм решения

3.1 Масштабирование

В общем случае масштабирование производится с помощью изменения количества реплик в нужном объекте Deployment с использованием API Kubernetes. Это изменение может быть вызвано либо оповещением от сервера метрик, либо задачей планировщика.

3.1.1 Сервер метрик и оповещения

Автоскейлер принимает решение о масштабировании на основании прочитанных метрик. Для этого он формирует запросы к серверу метрик, которые должны возвращать значения нагрузки модулей, и по расписанию собирает их.

Сервер метрик следит за состоянием кластера и развернутых на нем приложений. Он собирает информацию об использовании ресурсов машины, о входящих и исходящих HTTP-запросах и о многих других параметрах.

Также, такие сервера могут автоматически отправлять оповещения. Эта возможность используется для гибернации сервиса при отсутствии запросов на API в течение заданного времени.

3.1.2 Масштабирование с пользовательскими параметрами

Пользователь может указать параметры масштабирования для каждого сервиса в объекте конфигурации ConfigMap. В таком случае, максимальное количество реплик может превышать 1, и гибернация сервиса будет опциональной при минимальном количестве реплик, не равном 0.

Масштабирование производится по следующему алгоритму:

1. Проверка значения метрики в сервере метрик.
2. Если пороговое значение метрики не превышено, процесс завершается до следующей проверки. Иначе:
3. Поиск объектов Service, Deployment и Ingress, относящихся к сервису.
4. Изменение количества реплик в объекте Deployment до значения, указанного пользователем.

3.1.3 Конфигурация масштабирования

Конфигурация записывается в объект ConfigMap в формате YAML.

```
1 # ConfigMap для конфигурации масштабирования
2 apiVersion: v1
3 kind: ConfigMap
4 metadata:
5   name: autoscaler-props
6   namespace: unip-system-autoscaler
7 data:
8   spec.yaml: |
9     <текст spec.yaml>
```

Файл spec.yaml содержит параметры масштабирования для каждого сервиса, разделенные тремя дефисами.

Структура этого файла валидируется с помощью JSON Schema, которая позволяет четко задать формат настроек (см. файл unip_autoscaler/autoscaling_config.py).

В конфигурации есть несколько основных частей:

- target - целевой объект масштабирования. Для каждого приложения, которое нужно масштабировать, указывается его имя и пространство имен. В поле kind указывается тип объекта - Deployment, Service или MLComponent.
- scalingRules - правила скейлинга. Здесь прописывается то, при каких условиях и на сколько увеличивать / уменьшать количество реплик. Поддерживается либо упрощенный вариант конфигурации (по средней загрузке CPU и RAM в среднем по всем репликам), либо использование заданной пользователем метрики. Масштабирование выполняется при превышении прописанного порогового значения.
- timeWindow - промежуток, за который будет собираться и усредняться метрика (то есть, за какой промежуток времени до текущего момента считать среднее значение). Используется только в случае задания пороговых значений CPU и RAM - предполагается, что пользовательский запрос будет сам включать временной промежуток и возвращать вещественное число.

- maxReplicas - максимальное количество реплик.
- minReplicas - минимальное количество реплик. При значении 0 доступна гибернация.
- cooldown - время в секундах, после которого не производится масштабирование.

```

1  # Пример spec.yaml
2  target:
3    kind: deployment
4    name: student-chatbot-mlcmp-deployment
5    namespace: pu-test-pa-student-chatbot
6  scalingRules:
7    scaleUp:
8      step: 2 # сколько реплик добавлять
9      # нужно указать хотя бы один из порогов cpuThreshold, memoryThreshold
10     # ЛИБО метрику из Prometheus
11     # Prometheus-запрос для этого соберется автоматически
12     cpuThreshold: 50 # пороговое использование CPU, %
13     memoryThreshold: 500 # пороговое использование памяти, МБ
14   scaleDown:
15     step: 1 # сколько реплик убирать
16     cpuThreshold: 10
17     memoryThreshold: 200
18   timeWindow: 300 # время в секундах, в течение которого собираются метрики
19   maxReplicas: 10 # максимальное количество реплик
20   minReplicas: 0 # минимальное; параметр minReplicas=0 включает гибернацию
21   cooldown: 300 # время в секундах, после которого возможно повторное масштабирование
22   ---
23   target:
24     kind: deployment
25     name: pythonserver
26     namespace: unip-system-autoscaler
27   scalingRules:
28     prometheusMetric:
29       query: |
30         sum by (pod) (rate(
31           container_cpu_usage_seconds_total{pod=~"pythonserver-.*"}[1m]
32         )) * 100
33     thresholdUp: 50 # пороговое значение, при превышении которого сервис масштабируется вверх
34     thresholdDown: 25 # то же самое, но вниз
35     stepUp: 1 # на сколько реплик увеличивать
36     stepDown: 1 # на сколько реплик уменьшать
37   maxReplicas: 3
38   minReplicas: 1
39   cooldown: 120

```

3.1.4 Гибернация и восстановление

В режиме работы по умолчанию гибернация производится после оповещения от AlertManager (менеджера оповещений) об отсутствии запросов на API интеллектуального сервиса в течение заданного времени.

Количество реплик уменьшается до 0, и следующий запрос переадресуется в автоскейлер.

Гибернация производится по следующему алгоритму:

1. Получение запроса на гибернацию сервиса.
2. Поиск объектов Service, Deployment и Ingress, относящихся к сервису.
3. Поиск объекта Service с суффиксом «-hibernated»; при отсутствии, его создание.
4. Изменение объекта Ingress для перенаправления HTTP-запросов на автоскейлер.
5. Изменение объекта Deployment для уменьшения количества реплик до 0.

Восстановление производится по следующему алгоритму:

1. Получение запроса на восстановление в сервис с суффиксом «-hibernated».

- Поиск объектов Service, Deployment и Ingress, относящихся к сервису.
- Изменение количества реплик в объекте Deployment до изначального значения.
- Запуск цикла для проверки готовности сервиса, во время которого перехваченный HTTP-запрос будет удерживаться, и при истечении таймаута будет возвращаться перенадресация со статусом 307 Temporary Redirect на тот же URL.
- Изменение объекта Ingress до изначального состояния.
- Перенадресация запроса на восстановленный сервис.

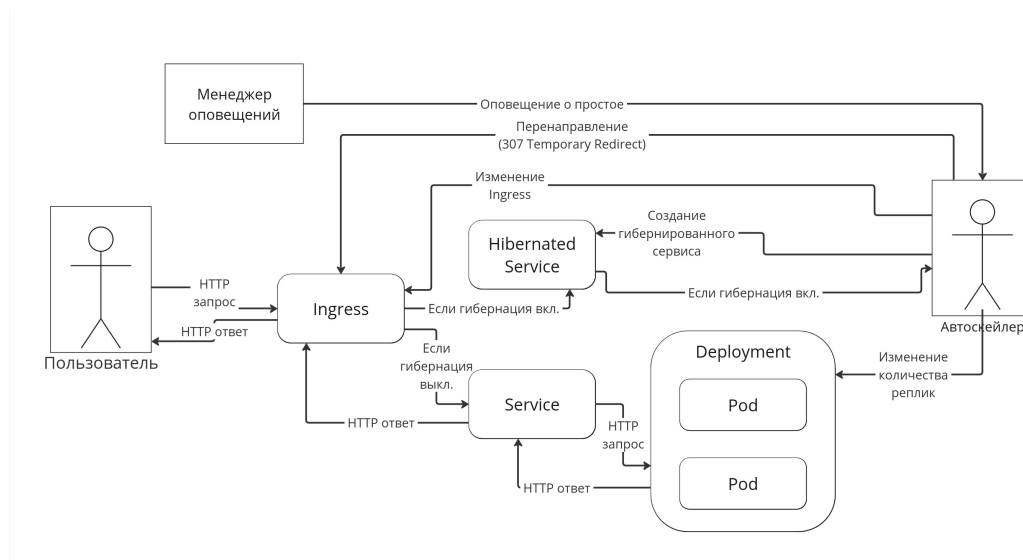


Рис. 2: Структурная схема гибернации и восстановления

4 Тестирование

4.1 Протокол тестирования

Для проверки работы модуля были проведены тесты, которые включали в себя:

- Тестирование гибернации и восстановления MLComponent с проверкой запросов на APIComponent
- Тестирование масштабирования Deployment стресс-теста с заданными порогами CPU и памяти (и отдельно с запросом в сервер метрик).

4.1.1 Тестирование гибернации и восстановления

- Запустить MLComponent с рабочей функцией inference (см. репозиторий `mlops_platform/demo-project`).
- Подождать определенное время до гибернации (по умолчанию 5 минут).
- Проверить, что в Ingress сервиса записались дополнительные правила для перенаправления на автоскейлер.
- Проверить, что создан сервис-заглушка с суффиксом «-hibernated».
- Проверить, что количество реплик Deployment равно 0.
- Послать запрос на инференс через Postman (или другой HTTP-клиент).
- Убедиться, что в логах автоскейлера появилось сообщение о восстановлении.
- Подождать, пока сервис не станет доступен и проверить количество реплик.
- Проверить, что результат HTTP-запроса верный.

4.1.2 Тестирование масштабирования Deployment

1. Запустить тестовый Deployment с приложением stress-ng (см. файл k8s/stress-ng.yaml).
2. Дождаться обновления значений CPU/RAM в сервере метрик.
3. Проверить, что количество реплик увеличилось до заданного значения.
4. Дождаться окончания работы стресс-теста во всех репликах.
5. Проверить, что количество реплик уменьшилось до минимального значения.
6. Повторить шаги выше с запросом в сервер метрик и приложением на Python (см. файл k8s/python.yaml).

4.2 Параметры тестирования

В процессе тестирования для сборки образов использовались следующие версии:

- Python версии 3.12
- Kubernetes версии 1.29
- Операционная система: Ubuntu 20.04

5 Ссылки на репозитории

- Репозиторий с исходным кодом модуля автоскейлинга: [ispirin/unip-autoscaler](#)
- Репозиторий с тестовым модулем: [mlops_platform/demo-project](#)